# HƯỚNG DẪN GIẢNG VIÊN — 09/10/2026

Tài liệu đáp án và điều hành buổi thực hành; phát bộ sinh viên riêng.

Mạch ứng dụng và dữ liệu trùng bài thực hành chính. Không yêu cầu sinh viên cài lại tất cả cấu trúc dữ liệu trong một buổi.

Tên tài liệu: Hướng dẫn giảng viên: Thực hành điều phối giao đơn bằng Julia

Ngày thực hành: 09/10/2026

Đối tượng sử dụng: Giảng viên; tách khỏi bộ phát sinh viên

Thời lượng đề xuất: 180

## Lưu ý điều hành

- Đây là một buổi thực hành của một ứng dụng. Chỉ đánh giá bốn hàm sinh viên tự viết: merge_sort, binary_search, bfs_routes và knapsack. Danh sách, FIFO, ngăn xếp, BST, DFS, đệ quy và vét cạn đã cung cấp để sinh viên thấy sự phối hợp; không biến việc đọc mã hỗ trợ thành bảy đề mới.
- Giữ thứ tự nhiệm vụ sort → search → BFS → DP vì build_session cần merge_sort để chuẩn hóa danh sách kề. Sau BFS, tái dùng merge_sort cho thứ tự ưu tiên theo lợi ích/τ. Nếu giới thiệu BFS trước bằng hình, dùng đồ thị mẫu đã cho và không chạy build_session khi hàm sort còn thiếu.
- Trước giờ học, kiểm tra Julia chạy được, khung sinh viên mở được và chế độ --data tạo MINI001 khi bốn TODO còn thiếu. Chuẩn bị máy giảng viên có mã lời giải riêng để trình diễn, nhưng không đặt lời giải trong bộ sinh viên.
- Phân biệt kết quả minh họa đã biết với kiểm thử chương trình. Một hoạt hình đẹp hoặc một menu mở được chưa chứng minh bốn hàm đúng. Chỉ ghi đạt sau chạy đúng pha kiểm thử và đối chiếu đầu ra thực tế.
- Mốc hỗ trợ: sau NV1 cần dãy mới [1,2,3]; sau NV2 phải tra Order/nothing; sau NV3 cần distance và routes hợp lệ cùng costs=Dict(1=>3,2=>5,3=>7); sau NV4 mới chạy ứng dụng trọn luồng.
- Bảng ánh xạ bảy chương là phụ lục giải thích vai trò; một buổi không hiện thực tất cả 29 bài của học phần. Mục tiêu là bốn thuật toán trọng tâm và sự nối kết đầu ra–đầu vào.
- MSSV chỉ dùng phần phát triển sau bộ mini trong phương án 180 phút. Không ép sinh viên vừa học chỉ số bảng DP vừa xử lý ngay bộ ngẫu nhiên 8–10 đơn trước khi kiểm tra được ba đơn bằng tay.
- Phần trình bày bằng Julia là bắt buộc. Trong lớp tập trung bốn hàm và kiểm thử; sau buổi học sinh viên hoàn thiện báo cáo cùng notebook Julia/Pluto mẫu gọi cùng lõi. Máy chưa cài Pluto được chạy trực tiếp tệp notebook .jl bằng Julia. Không tăng số TODO hoặc kéo dài kịch bản trên lớp.

## Đáp án câu hỏi cuối buổi

| Mã | Đáp án | Điểm |
|---|---|---|
| Q1 | Tìm kiếm nhị phân theo ID chỉ đúng khi danh mục có ID tăng, khóa duy nhất và không đổi trong lượt tra cứu. Dãy lợi ích/τ có thứ tự theo khóa khác; nhánh loại trái/phải dựa ID không còn hợp lệ. | 2 |
| Q2 | Điểm 4 cách nguồn 1 hai cạnh; lượt đi và về cần 2×2=4, phục vụ 1 nên τ=5. BFS tối ưu số cạnh trong đồ thị không trọng số hoặc cùng trọng số. Với thời gian cạnh khác nhau, số cạnh ít nhất không nhất thiết có tổng thời gian ít nhất; không dùng BFS thông thường để kết luận tối ưu tổng trọng số. | 2 |
| Q3 | Không lấy đơn 2: F(1,8)=6. Lấy đơn 2: lợi ích 8+F(1,3)=14. F(2,8)=14, ánh xạ table[3,9]. Nhánh lấy dùng hàng trước để phần còn lại chỉ xét các đơn 1..i−1; đọc cùng hàng có thể chọn lại đơn i và biến thành bài ba lô không giới hạn. | 2 |
| Q4 | Sau giao 1: FIFO [2], lịch sử [1], thời gian 3, lợi ích 6. Hoàn tác ngay: FIFO [1,2], lịch sử [], thời gian 0, lợi ích 0. ID chờ và đã giao không giao nhau, hợp đúng tập chọn, các tổng bằng tổng trên ID đã giao. Đưa đơn về cuối FIFO không khôi phục đúng thứ tự ban đầu. | 2 |

| Mã | Đáp án | Điểm |
|---|---|---|
| Q5 | Sắp xếp trộn O(n log n); nhị phân O(log n); BFS cốt lõi O(V+E); DP bảng đầy đủ O(nB) thời gian và bộ nhớ. BST tra cứu O(h), có thể lệch với h=O(n). Khi BFS lưu routes cho mọi đỉnh, tính thêm tổng độ dài đường và bộ nhớ đường, có thể $O(V^2)$; không nhầm bộ nhớ này với BFS chỉ lưu parent. | 2 |

## Dữ liệu và kết quả bộ mini

Ngân sách: 8

Số đỉnh: 6

## Cạnh bản đồ

- 1
- 2
- 1
- 3
- 2
- 4
- 3
- 4
- 3
- 5
- 4
- 6
- 5
- 6

## Đơn hàng

| Mã | Điểm giao | Phục vụ | Lợi ích | Số cạnh | Chi phí | Tuyến |
|---|---|---|---|---|---|---|
| 1 | 2 | 1 | 6 | 1 | 3 | [1, 2] |

| Mã | Điểm giao | Phục vụ | Lợi ích | Số cạnh | Chi phí | Tuyến |
|---|---|---|---|---|---|---|
| 2 | 4 | 1 | 8 | 2 | 5 | [1, 2, 4] |
| 3 | 6 | 1 | 10 | 3 | 7 | [1, 2, 4, 6] |

### distance

- 0
- 1
- 1
- 2
- 2
- 3

### priority

- 1
- 2
- 3

### Tập chọn

- 1
- 2

Chi phí: 8

Lợi ích: 14

### subsets

| ids | Chi phí | Lợi ích | feasible |
|---|---|---|---|
| [] | 0 | 0 | True |
| [1] | 3 | 6 | True |
| [2] | 5 | 8 | True |
| [3] | 7 | 10 | True |
| [1, 2] | 8 | 14 | True |
| [1, 3] | 10 | 16 | False |

| ids | Chi phí | Lợi ích | feasible |
|---|---|---|---|
| [2, 3] | 12 | 18 | False |
| [1, 2, 3] | 15 | 24 | False |

## Giải thích bảng quy hoạch động

F(i,b) là lợi ích tối đa từ i đơn đầu với ngân sách b; Julia lưu ở table[i+1,b+1].

| i / b | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 |
|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 1 | 0 | 0 | 0 | 6 | 6 | 6 | 6 | 6 | 6 |
| 2 | 0 | 0 | 0 | 6 | 6 | 8 | 8 | 8 | 14 |
| 3 | 0 | 0 | 0 | 6 | 6 | 8 | 8 | 10 | 14 |

- F(2,8): không lấy đơn 2 được 6; lấy được 8+F(1,3)=14; chọn 14.
- F(3,8): không lấy đơn 3 được 14; lấy được 10+F(2,1)=10; chọn 14.
- F(3,7): không lấy đơn 3 được 8; lấy được 10+F(2,0)=10; chọn 10. Ô này giúp sinh viên thấy quyết định phụ thuộc ngân sách.

| i | b | Quyết định |
|---|---|---|
| 3 | 8 | F(3,8)=F(2,8)=14; không lấy ID 3; chuyển i=2,b=8. |
| 2 | 8 | F(2,8)=14 lớn hơn F(1,8)=6; lấy ID 2; chuyển i=1,b=3. |
| 1 | 3 | F(1,3)=6 lớn hơn F(0,3)=0; lấy ID 1; chuyển i=0,b=0 và dừng. |

Khi giá trị bằng nhau, có thể không lấy hoặc chọn theo quy tắc khác nếu truy vết vẫn hợp lệ. Không chấm bằng so sánh cứng toàn bộ IDs với một lời giải khi có nhiều nghiệm tối ưu.

## Truy vết ứng dụng

| Bước | Tập chọn | Hàng đợi | Lịch sử | Thời gian đã dùng | Lợi ích đã nhận |
|---|---|---|---|---|---|
| Lập kế hoạch | [1, 2] | [1, 2] | [] | 0 | 0 |
| Giao ID 1 | [1, 2] | [2] | [1] | 3 | 6 |

| Bước | Tập chọn | Hàng đợi | Lịch sử | Thời gian đã dùng | Lợi ích đã nhận |
|---|---|---|---|---|---|
| Hoàn tác | [1, 2] | [1, 2] | [] | 0 | 0 |
| Giao lại ID 1 | [1, 2] | [2] | [1] | 3 | 6 |
| Giao ID 2 | [1, 2] | [] | [1, 2] | 8 | 14 |

## Ví dụ đổi bản đồ

purpose: Kiểm tra BFS thật sự cung cấp chi phí cho DP; chỉ dùng bản sao dữ liệu thử, giữ nguyên CSV chính thức.

## removedEdges

- 2
- 4
- 3
- 4

## remainingEdges

- 1
- 2
- 1
- 3
- 3
- 5
- 4
- 6
- 5
- 6

Ngân sách: 8

## Chi phí

- 3
- 9

- 7

### routeForOrder2

- 1
- 3
- 5
- 6
- 4

### Tập chọn

- 3

Chi phí: 7

Lợi ích: 10

explanation: Đồ thị vẫn liên thông. Đơn 2 cần bốn cạnh tới điểm 4, nên τ=2×4+1=9; không còn vừa B=8. Đơn 1 và 3 giữ τ=3,7. Tối ưu đổi từ {1,2} lợi ích 14 sang {3} lợi ích 10. Nếu vẫn chọn {1,2}, cần tìm chi phí gán tay hoặc từ điển costs cũ trong luồng.

### Phản ví dụ của tham lam

Ngân sách: 13

### Đơn hàng

| Mã | Điểm giao | Phục vụ | Lợi ích | Chi phí |
|---|---|---|---|---|
| 1 | 2 | 3 | 9 | 5 |
| 2 | 4 | 2 | 10 | 6 |
| 3 | 6 | 1 | 10 | 7 |

### greedy

### ids

- 1
- 2

Chi phí: 11

Lợi ích: 19

### optimal

### ids

- 2
- 3

Chi phí: 13

Lợi ích: 20

explanation: Tỷ số 9/5 > 10/6 > 10/7 khiến tham lam chọn 1 rồi 2. Tập 2,3 cho lợi ích cao hơn. Dùng khi sinh viên hỏi vì sao cần DP dù tham lam đúng ở ví dụ mini.

## Phương án 90 phút

| Số phút | Tên tài liệu | Hoạt động | Minh chứng |
|---|---|---|---|
| 10 | Đặc tả và dữ liệu | Đọc cùng hai CSV mini, tính tay τ và xem sơ đồ một ứng dụng. | Dữ liệu và công thức 3,5,7. |
| 15 | Hiện thực merge_sort | Làm NV1 và thử pha sort; giảng viên rút gọn truy vết đệ quy. | Dãy ID tăng, đủ phần tử, không đổi đầu vào. |
| 15 | Hiện thực binary_search | Làm NV2 và thử pha search trên danh mục đã sắp. | Order/nothing và bảng biên tìm kiếm. |
| 15 | BFS có hướng dẫn | Giảng viên cung cấp phiên bản BFS đúng sau phần dự đoán FIFO; sinh viên điền parent/distance và giải thích τ. Không đánh giá như sinh viên đã tự viết BFS. | Khoảng cách và đường đi đúng, điều kiện áp dụng BFS. |
| 15 | DP và truy vết có hướng dẫn | Giảng viên dùng phiên bản DP đúng; sinh viên tính F(2,8), F(3,8) và truy vết. NV3/NV4 là bài hoàn thiện sau buổi. | Chọn 1,2 và giải thích được quyết định lấy/không lấy. |
| 10 | Ứng dụng trọn luồng | Gọi cùng lõi đã hoàn chỉnh, giao–hoàn tác–xuất và kiểm tra trạng thái. | Một lượt chạy tích hợp, các tổng khớp. |

| Số phút | Tên tài liệu | Hoạt động | Minh chứng |
|---|---|---|---|
| 10 | Quiz và nộp tiến độ | Trả lời câu hỏi ngắn, ghi phần tự làm/phần được cung cấp và commit tiến độ. | URL/commit, trạng thái hoàn thiện rõ ràng. |

## Phương án 225 phút

| Số phút | Tên tài liệu | Hoạt động | Minh chứng |
|---|---|---|---|
| 10 | Đặc tả và luồng ứng dụng | Giới thiệu bài toán, đầu vào/đầu ra và sơ đồ xuyên suốt. | Một sơ đồ luồng. |
| 15 | Tạo và đọc CSV | Tạo MINI001, đối chiếu dữ liệu và các điều kiện. | Hai CSV và mô tả cột. |
| 25 | NV1: Sắp xếp trộn | Hoàn thiện hàm, truy vết, kiểm thử pha sort. | Dãy mới đúng, trường hợp biên đạt. |
| 20 | NV2: Tìm kiếm nhị phân | Hoàn thiện hàm, bảng biên, kiểm thử pha search. | Order/nothing, đối chiếu BST. |
| 30 | NV3: BFS | Hoàn thiện FIFO/parent/routes, kiểm thử và tính costs. | Khoảng cách, tuyến đường và 3,5,7. |
| 35 | NV4: Quy hoạch động | Hoàn thiện bảng và truy vết, đối chiếu vét cạn. | Giá trị 14, tập hợp lệ và ô giải thích. |
| 20 | Tích hợp giao–hoàn tác–xuất | Chạy trọn luồng và kiểm tra bất biến. | Các tổng và CSV khớp trạng thái. |
| 25 | Dữ liệu MSSV và truy vết cá nhân | Sinh CSV theo MSSV, chạy cùng ứng dụng, theo dõi một ID thực tế; không sửa bộ dữ liệu. | Một chuỗi CSV→BFS→DP→trạng thái trên dữ liệu cá nhân. |
| 20 | Thử bản đồ thay đổi và phản ví dụ tham lam | Dùng bản sao thử bỏ hai cạnh để thấy DP đổi theo BFS; đối chiếu phản ví dụ 19<20 nếu đủ thời gian. | Costs 3,9,7, chọn ID3; lý giải phạm vi tham lam. |
| 10 | Quiz | Trả lời độc lập năm câu cuối buổi. | Phiếu trả lời có lý giải. |

| Số phút | Tên tài liệu | Hoạt động | Minh chứng |
|---|---|---|---|
| 15 | Ghi kết quả và GitHub | Chạy kiểm thử all, ghi kết quả, commit và lập dàn ý báo cáo; hoàn thiện báo cáo cùng notebook Julia mẫu sau buổi học. | Lệnh chạy, kết quả thật, URL/commit và dàn ý báo cáo; notebook bắt buộc hoàn thiện sau buổi. |

## Hướng dẫn chấm

- Rubric sinh viên tổng 100; đây là thang đề xuất cho buổi thực hành, giảng viên quyết định cách quy đổi vào điểm học phần. Năm câu quiz dùng phát vấn hoặc phần trình bày, không cộng một thang 10 riêng làm tổng thành 110.
- Chấm tính đúng của từng hàm độc lập bằng các pha kiểm thử, sau đó chấm tích hợp. Lỗi ở một hàm đầu không tự động xóa điểm ý tưởng hoặc hàm sau nếu sinh viên có kiểm thử riêng đáng tin cậy.
- NV1: kiểm tra đủ phần tử, không đổi đầu vào và ổn định. NV2: đúng Order/nothing và tiền điều kiện. NV3: đúng khoảng cách, routes hợp lệ, xử lý unreachable và costs nối DP. NV4: bảng đúng, tập hợp lệ, không lặp và truy vết khớp value.
- Không chấm thiếu chỉ vì một đường BFS khác đường mẫu nhưng có cùng độ dài. Với DP nhiều nghiệm, so sánh tính khả thi và giá trị; thứ tự hàng đợi mới cần đúng quy tắc ưu tiên.
- Yêu cầu sinh viên chỉ vào mã và giải thích một thay đổi trạng thái. Một bản code không giải thích được chưa đủ bằng chứng cho điểm trình bày ý tưởng.
- Báo cáo chỉ cần 3–5 trang khuyến nghị; ưu tiên bảng thật, sơ đồ và lý giải, không ưu tiên độ dài. Chỉ ghi đạt/chưa đạt dựa lượt chạy thực tế. Báo cáo và notebook Julia mẫu hoàn thiện sau lớp; notebook là phần trình bày bắt buộc, có thể mở bằng Pluto hoặc chạy trực tiếp .jl nếu máy chưa cài Pluto.
- Với phương án 90 phút, công bố rõ phạm vi trên lớp chỉ tự hiện thực NV1/NV2; NV3/NV4 hoàn thiện sau buổi. Không dùng kết quả giảng viên cung cấp để ghi sinh viên đã hiện thực đủ bốn hàm trong 90 phút.

## Câu hỏi trao đổi

| Câu hỏi | lookFor |
|---|---|
| Nếu tra cứu ID99 không tồn tại, kế hoạch hoặc FIFO có thay đổi không? | Không; tra cứu là thao tác chỉ đọc và trả nothing. |
| Vì sao DP phải tra costs[o.id] thay vì lấy vị trí i làm khóa? | costs là Dict theo ID; ID không nhất thiết bằng vị trí sau sắp xếp, chi phí phải ghép với đúng bản ghi. |
| Có thể chạy BFS một lần thay vì n lần không? | Có, mọi đơn cùng nguồn1; lấy distance ở từng điểm giao. |

| Câu hỏi | lookFor |
| --- | --- |
| Bỏ hai cạnh (2,4),(3,4) rồi chương trình vẫn chọn1,2: nên kiểm tra gì trước? | Chi phí cũ/gán tay và việc kết quả BFS thực sự truyền vào DP; chi phí đúng của đơn2 nay là9. |
| Hoàn tác lần mới nhất vì sao dùng ngăn xếp? | LIFO giữ đúng đảo ngược thao tác gần nhất; đưa ID về đầu FIFO để khôi phục thứ tự trước giao. |
| Bảng DP dùng O(nB); có phải đa thức theo số bit biểu diễn B không? | Không; theo giá trị B, thuộc phân tích giả đa thức; không buộc sinh viên dùng thuật ngữ nếu chưa học nhưng cần hiểu giới hạn khi B rất lớn. |

## Lệnh chạy dành cho giảng viên

- julia TH_2026_10_09_Kiem_thu.jl --teacher --phase=all
- julia TH_2026_10_09_Loi_giai.jl --data
- julia TH_2026_10_09_Loi_giai.jl --mini
- julia TH_2026_10_09_Loi_giai.jl N24DCDT040 --data
- julia TH_2026_10_09_Loi_giai.jl N24DCDT040 --demo

## Kiểm tra tài liệu trước buổi học

- Bộ sinh viên chứa đề, khung TODO, kiểm thử và dữ liệu; không chứa teacher_notes hoặc mã lời giải.
- Bốn chữ ký hàm, hợp đồng kết quả trả về, cờ --phase và tên file trong tài liệu phải khớp mã thực tế.
- Chạy lời giải trên dữ liệu mini và cá nhân; chạy các trường hợp biên có ý nghĩa; kiểm thử khung còn TODO phải báo chưa hoàn thiện rõ ràng.
- Kiểm tra tổng timeline180,90,225; rubric100; đúng ngày09/10/2026; quiz sinh viên không có đáp án.
- Kiểm tra tiếng Việt, font trình chiếu, bảng DP, ảnh/hình SVG và điều khiển hoạt hình; bản in vẫn đọc được khi không chạy JavaScript.

# Code tham khảo bốn hàm trọng tâm

## merge_sort

```
function merge_sort(a::AbstractVector, before::Function)
    length(a) <= 1 && return collect(a)
    mid = div(length(a),2)
    left = merge_sort(a[1:mid],before)
    right = merge_sort(a[mid+1:end],before)
    out = eltype(a)[]
    i = j = 1
    while i <= length(left) && j <= length(right)
        if before(right[j],left[i])
            push!(out,right[j]); j += 1
        else
            push!(out,left[i]); i += 1
        end
    end
    while i <= length(left); push!(out,left[i]); i += 1; end
    while j <= length(right); push!(out,right[j]); j += 1; end
    out
end
```

## binary_search

```
function binary_search(a::Vector{Order},id::Int)
    lo,hi = 1,length(a)
    while lo <= hi
        mid = lo + div(hi-lo,2)
        a[mid].id == id && return a[mid]
        if a[mid].id < id; lo = mid+1; else; hi = mid-1; end
    end
    nothing
end
```

## bfs_routes

```
function bfs_routes(graph,source=1)
    n = length(graph)
    distance = fill(-1,n); parent = zeros(Int,n)
    q = CircularQueue(n)
    distance[source] = 0; enqueue!(q,source)
    while q.count > 0
        u = dequeue!(q)
        for v in graph[u]
            if distance[v] == -1
                distance[v] = distance[u]+1
                parent[v] = u
                enqueue!(q,v) || error("Hàng đợi BFS quá tải.")
            end
        end
    end
    routes = [Int[] for _ in 1:n]
    for v in 1:n
        distance[v] == -1 && continue
        route = Int[]; cursor = v
        while cursor != 0
            push!(route,cursor); cursor = parent[cursor]
        end
        routes[v] = reverse(route)
    end
    (distance=distance,parent=parent,routes=routes)
end
```

### knapsack

```
function knapsack(orders,costs,budget)
    budget >= 0 || throw(ArgumentError("Ngân sách không được âm."))
    a = merge_sort(orders,(x,y)->x.id < y.id)
    n = length(a); table = zeros(Int,n+1,budget+1)
    for i in 1:n, b in 0:budget
        o = a[i]; cost = costs[o.id]
        cost > 0 || throw(ArgumentError("Chi phí đơn phải dương."))
        skip = table[i,b+1]
        take = cost <= b ? table[i,b-cost+1] + o.benefit : -1
        table[i+1,b+1] = max(skip,take)
    end
    ids = Int[]; b = budget
    for i in n:-1:1
        if table[i+1,b+1] > table[i,b+1]
            push!(ids,a[i].id); b -= costs[a[i].id]
        end
    end
    ids = reverse(ids)
    (ids=ids,value=table[n+1,budget+1],cost=budget-b,table=table)
end
```

# Chạy lời giải và kiểm thử

```
julia TH_2026_10_09_Loi_giai.jl --mini
julia TH_2026_10_09_Kiem_thu.jl --teacher --phase=all
```